

6.AR vision

1. Overview

Augmented Reality, referred to as "AR", is a technology that skillfully integrates virtual information with the real world. Extensive use of multimedia, three-dimensional modeling, real-time tracking and registration, intelligent interaction, sensing and other technical means to simulate computer-generated text, images, three-dimensional models, music, videos and other virtual information. When applied to the real world, the two types of information complement each other, thereby achieving "enhancement" of the real world.

The AR system has three outstanding characteristics: ① information integration between the real world and the virtual world; ② real-time interactivity; ③ adding positioning virtual objects in the three-dimensional scale space.

Augmented reality technology includes new technologies and methods such as multimedia, three-dimensional modeling, real-time video display and control, multi-sensor fusion, real-time tracking and registration, and scene fusion.

2. How to use

When using AR applications, camera intrinsic parameters are essential; otherwise, the application will not run. The intrinsic parameter file should be in the same directory as the code, and different cameras require different intrinsic parameters. Intrinsic parameter calibration can be quickly performed using a chessboard pattern; for detailed instructions, please refer to [Depth Camera Course]--[2.Internal reference calibration]. **(This step has already been completed in the Docker image.)**

After the calibration work is completed, a [calibrationdata.tar.gz] will be generated, and the [calibrationdata.tar.gz] file will be moved to the [home] directory. After decompression, open [ost.yaml] in the folder, find the camera internal parameter matrix and distortion coefficient and modify them to the corresponding locations of the [astra.yaml] file. You only need to modify the contents of two [data]. For example: the following,

```
camera_matrix: !!opencv-matrix
  rows: 3
  cols: 3
  dt: d
  data: [615.50506, 0. , 365.84388,
        0. , 623.69024, 238.778 ,
        0. , 0. , 1. ]
distortion_model: plumb_bob
distortion_coefficients: !!opencv-matrix
  rows: 1
  cols: 5
  dt: d
  data: [0.166417, -0.160106, -0.008776, 0.025459, 0.000000]
```

There are a total of 12 effects in the cases in this section.

```
["Triangle", "Rectangle", "Parallelogram", "WindMill", "TableTennisTable",
"Ball", "Arrow", "Knife", "Desk", "Bench", "Stickman", "ParallelBars"]
```

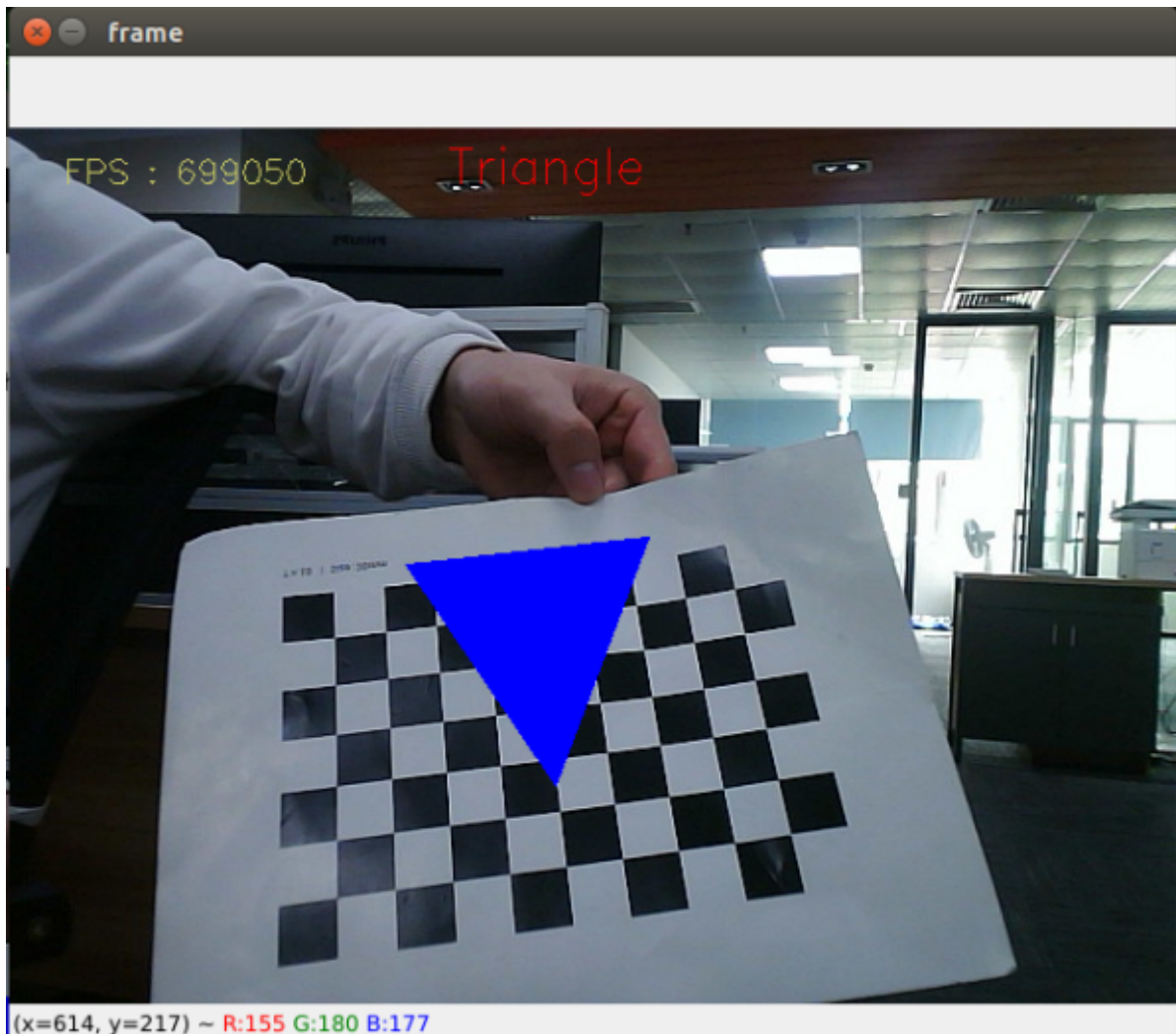
3. Start command

Code reference path,

```
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_visual/yahboomcar_visual/s  
imple_AR.py
```

After entering the docker container, enter the docker terminal,

```
ros2 run yahboomcar_visual simple_AR
```



[q] key to exit, [f] key to switch different effects.

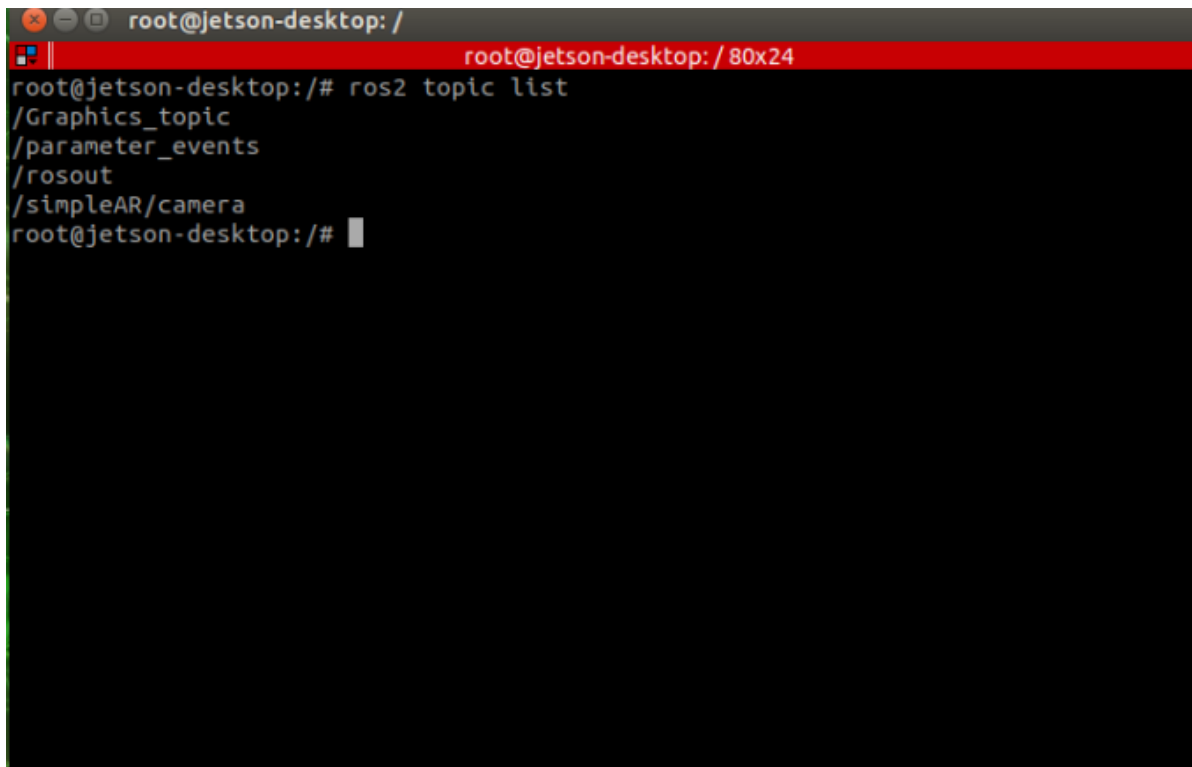
3.1.1. ROS deployment

This course also deploys ROS, which mainly has the following two functions:

- Subscribe to topic data and switch between different effects
- Post images

View the ros topic through the following command, enter in the docker terminal,

```
ros2 topic list
```

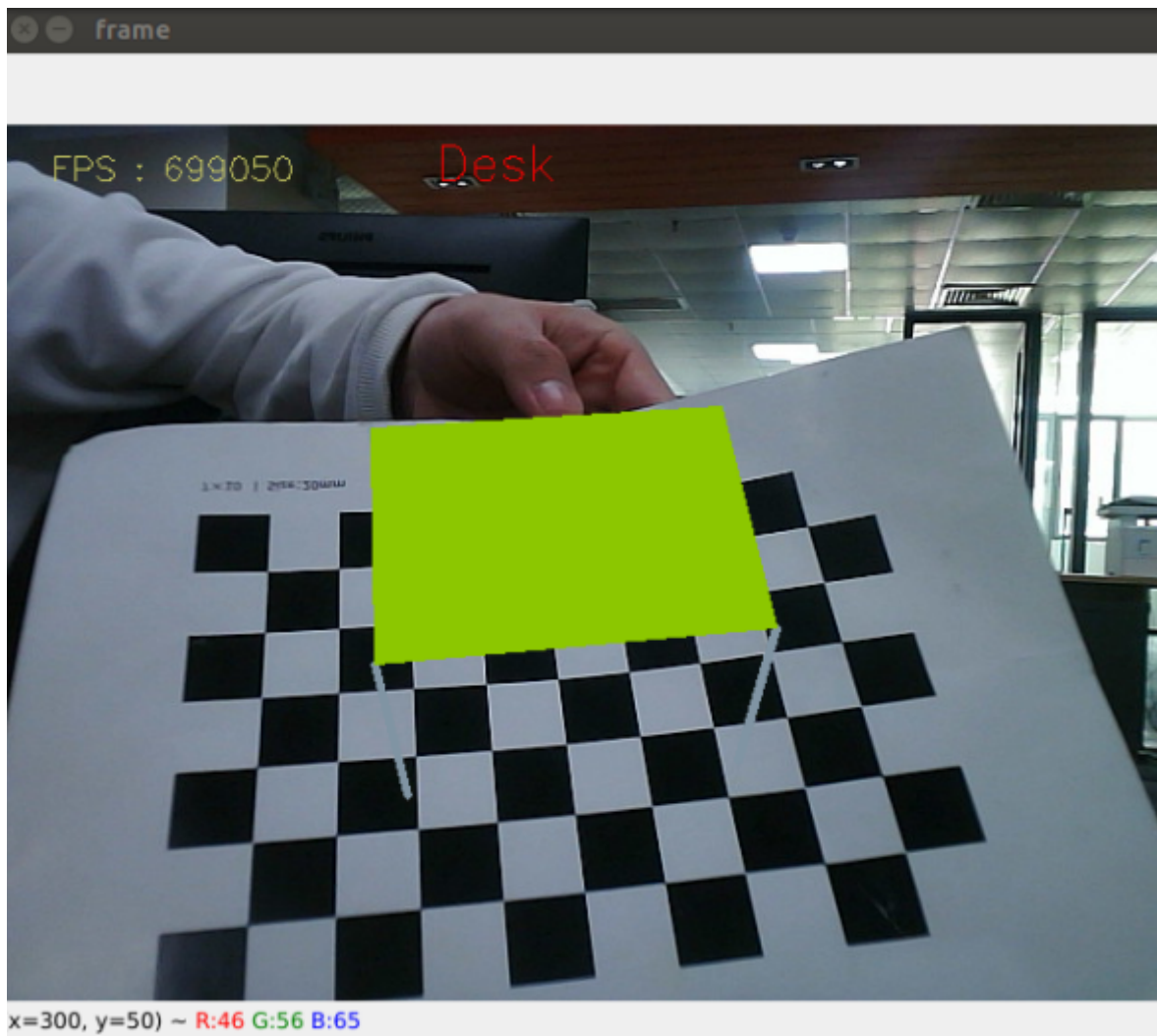
A terminal window titled 'root@jetson-desktop: /' with a red header bar containing 'root@jetson-desktop: / 80x24'. The terminal shows the command 'ros2 topic list' and its output: '/Graphics_topic', '/parameter_events', '/rosout', and '/simpleAR/camera'. The prompt 'root@jetson-desktop: /# ' is visible at the bottom.

```
root@jetson-desktop: /  
root@jetson-desktop: / 80x24  
root@jetson-desktop: /# ros2 topic list  
/Graphics_topic  
/parameter_events  
/rosout  
/simpleAR/camera  
root@jetson-desktop: /#
```

- /Graphics_topic: The topic name of the effect, the effect that needs to be identified when subscribing.
- /simpleAR/camera: The topic name of the image, publish the image.

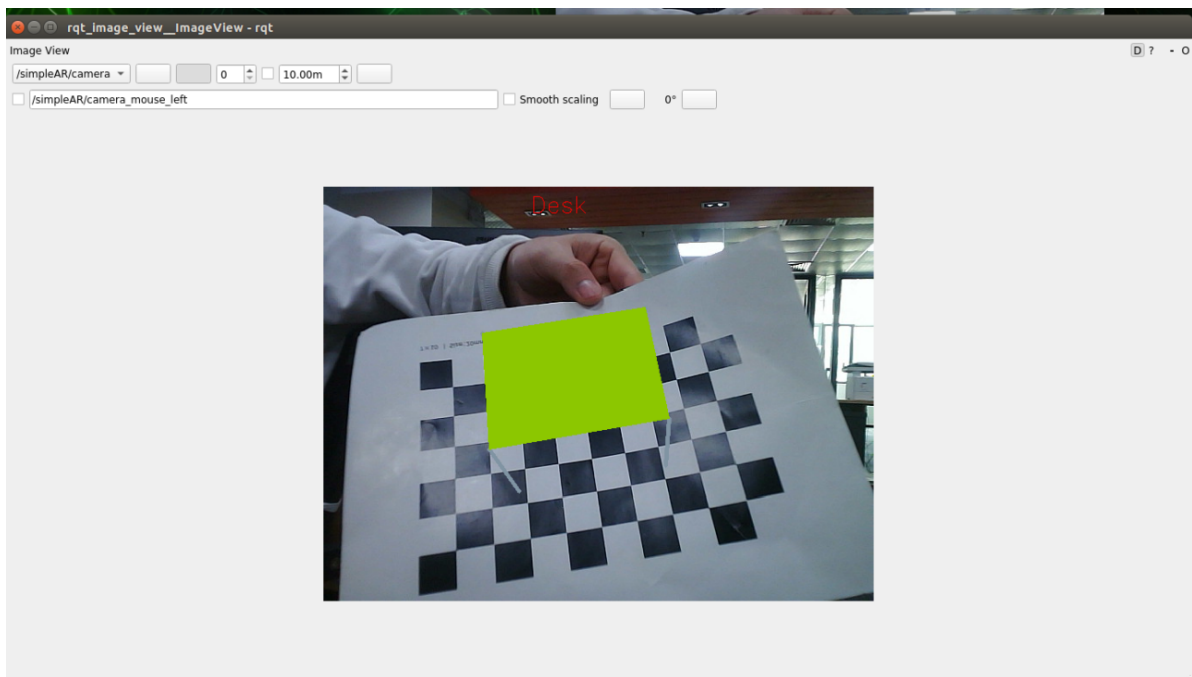
The modification effect can be modified through the following command. For example, I first changed it to Desk and entered it in the docker terminal.

```
ros2 topic pub /Graphics_topic std_msgs/msg/String "data: Desk"
```



To view the published image, you can use `rqt_image_view` to view it and enter it in the docker terminal.

```
ros2 run rqt_image_view rqt_image_view
```



Select the topic `/simpleAR/camera` in the upper left corner to view the image.

